

Exercício Prático: Sistema de Agendamento de Consultas

Turma: LionsDev

Tópicos: CRUD (Create, Read, Update, Delete), Arrays de Objetos, Modularização com `export default` / `import from`, Validação de Dados e Menu Interativo com `prompt-sync`.

1. Base de Dados

Para começar, utilize a estrutura de dados base apresentada em aula. Salve este conteúdo em um arquivo chamado `dados.js`:

```
const medicos = [
  { id: 1, nome: "Dr. House", especialidade: "Diagnóstico" },
  { id: 2, nome: "Dra. Grey", especialidade: "Cirurgia" },
];

const pacientes = [
  { id: 1, nome: "John Doe", dataNascimento: "1985-01-15" },
  { id: 2, nome: "Jane Smith", dataNascimento: "1990-05-30" },
];

let consultas = [
  { id: 1, data: "2023-01-10", idMedico: 1, idPaciente: 1, descricao: "Consulta inicial" },
  { id: 2, data: "2023-02-15", idMedico: 2, idPaciente: 1, descricao: "Seguimento" },
  { id: 3, data: "2023-03-20", idMedico: 1, idPaciente: 2, descricao: "Consulta de rotina" },
];

export default { medicos, pacientes, consultas };
```

2. Requisitos Obrigatórios

Crie um arquivo principal `index.js` contendo um menu interativo com o `prompt`, semelhante ao sistema de contatos, com as seguintes opções:

2.1 Agendar nova consulta (CREATE)

Crie uma função `adicionarConsulta` que receba os dados inseridos pelo usuário no terminal.

- **Regra 1:** O sistema deve gerar um `id` sequencial para a nova consulta automaticamente.

- **Regra 2 (Validação):** Antes de salvar a consulta, o sistema deve verificar se o `idMedico` digitado realmente existe no array de `medicos`. Faça o mesmo para o `idPaciente`. Se um dos dois não existir, exiba um erro e aborte o cadastro.

2.2 Listar todas as consultas (READ)

Crie uma função `listarConsultas` que percorra o array de consultas.

- **Regra:** O terminal não deve imprimir apenas os números de `idMedico` e `idPaciente`. Você deve usar lógicas de busca (como o `encontrarMedicoPorId` visto em aula) para imprimir o **Nome do Médico** e o **Nome do Paciente** ao lado da data e descrição.

2.3 Atualizar uma consulta (UPDATE)

Crie uma função `atualizarConsulta` que receba o `id` da consulta que o usuário deseja alterar.

- **Regra:** Permita que o usuário altere apenas a **data** e a **descrição** da consulta. Use a regra do operador lógico `||` ensinada em aula para manter o dado antigo caso o usuário deixe o campo em branco. Não permita alterar o médico ou o paciente (se errar isso, a regra de negócio diz que a consulta deve ser cancelada e refeita).

2.4 Cancelar consulta (DELETE)

Crie uma função `cancelarConsulta` que receba o `id` da consulta.

- **Regra:** Encontre o índice da consulta usando `.findIndex()` e remova-a do array utilizando o método `.splice()`. Imprima uma mensagem de sucesso ou uma mensagem de erro caso o ID fornecido não exista.

3. Dicas para a Implementação

- **Modularização:** Assim como feito nos slides, crie um arquivo `.js` separado para cada uma das quatro funções do CRUD e importe-os no seu menu principal.
 - **Funções Auxiliares:** Crie funções como `encontrarPacientePorId(id)` para evitar repetir o código do `for` ou `.find()` toda vez que precisar descobrir o nome de um paciente na hora de listar as consultas.
 - **Atenção aos Tipos:** Lembre-se que o dado que vem do `prompt` é sempre um texto (`String`). Se for comparar com os IDs do array (que são números), use `parseInt()` ou verifique usando `==` ao invés de `===`.
-

